

CS 229, Autunno 2018
Problem Set #1: Apprendimento Supervisionato
(Traduzione italiana ad uso di studio)

Note: (1) Queste domande richiedono ragionamento, ma non risposte lunghe: sii il più conciso possibile. (2) Per i problemi di programmazione non puoi usare librerie diverse da quelle definite nel file `environment.yml`; in particolare, librerie ML come scikit-learn non sono ammesse. (3) Il codice deve girare senza errori con lo script `run.py` e verrà valutato da un auto-grader su un test set privato.

Codice d'onore: è incoraggiato lavorare in gruppi di studio, ma ogni studente deve scrivere le soluzioni in autonomia, senza consultare appunti scritti della sessione di gruppo, e deve capire la soluzione abbastanza bene da poterla ricostruire da solo. È vietato consultare le soluzioni degli anni precedenti.

1. [40 punti] Classificatori lineari (regressione logistica e GDA)

In questo problema trattiamo due classificatori lineari probabilistici visti a lezione. Primo, un classificatore lineare *discriminativo*: la regressione logistica. Secondo, un classificatore lineare *generativo*: l'analisi discriminante gaussiana (GDA). Entrambi gli algoritmi trovano un confine di decisione lineare che separa i dati in due classi, ma partono da assunzioni diverse. L'obiettivo è capire più a fondo somiglianze e differenze (e punti di forza e debolezza) dei due algoritmi.

Considereremo due dataset, forniti nei file:

i. `data/ds1_{train,valid}.csv`

ii. `data/ds2_{train,valid}.csv`

Ogni file contiene m esempi, un esempio $(x^{(i)}, y^{(i)})$ per riga. In particolare, la riga i -esima contiene le colonne $x_0^{(i)} \in \mathbb{R}$, $x_1^{(i)} \in \mathbb{R}$ e $y^{(i)} \in \{0, 1\}$. Nei sottoproblemi che seguono useremo regressione logistica e GDA per fare classificazione binaria su questi due dataset.

(a) [10 punti]

A lezione abbiamo visto la perdita empirica media per la regressione logistica:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})),$$

dove $y^{(i)} \in \{0, 1\}$, $h_{\theta}(x) = g(\theta^T x)$ e $g(z) = 1/(1 + e^{-z})$.

Trova l'Hessiana H di questa funzione e mostra che, per ogni vettore z , vale $z^T H z \geq 0$.

Suggerimento: puoi iniziare mostrando che $\sum_i \sum_j z_i x_i x_j z_j = (x^T z)^2 \geq 0$. Ricorda inoltre che $g'(z) = g(z)(1 - g(z))$.

Osservazione: questo è uno dei modi standard per dimostrare che la matrice H è semidefinita positiva, scritto “ $H \succeq 0$ ”. Ciò implica che J è convessa e non ha minimi locali diversi da quello globale. Se conosci un altro modo per mostrare $H \succeq 0$, puoi usare il tuo metodo.

(b) [5 punti] Problema di programmazione

Segui le istruzioni in `src/p01b_logreg.py` per addestrare un classificatore a regressione logistica usando il metodo di Newton. Partendo da $\theta = \vec{0}$, esegui il metodo di Newton finché gli aggiornamenti di θ diventano piccoli: in particolare, addestra fino alla prima iterazione k tale che $\|\theta_k - \theta_{k-1}\|_1 < \varepsilon$, con $\varepsilon = 1 \times 10^{-5}$. Assicurati di scrivere le predizioni del modello nel file specificato nel codice.

(c) [5 punti]

Ricorda che nella GDA modelliamo la distribuzione congiunta di (x, y) con le equazioni:

$$p(y) = \begin{cases} \phi & \text{se } y = 1 \\ 1 - \phi & \text{se } y = 0 \end{cases}$$

$$p(x \mid y = 0) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1} (x - \mu_0)\right)$$

$$p(x \mid y = 1) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1} (x - \mu_1)\right),$$

dove ϕ , μ_0 , μ_1 e Σ sono i parametri del modello.

Supponi di avere già stimato ϕ , μ_0 , μ_1 e Σ , e di voler predire y dato un nuovo punto x . Per mostrare che la GDA produce un classificatore con confine di decisione lineare, dimostra che la distribuzione a posteriori può essere scritta come

$$p(y = 1 \mid x; \phi, \mu_0, \mu_1, \Sigma) = \frac{1}{1 + \exp\left(-(\theta^T x + \theta_0)\right)},$$

dove $\theta \in \mathbb{R}^n$ e $\theta_0 \in \mathbb{R}$ sono opportune funzioni di ϕ , Σ , μ_0 e μ_1 .

(d) [7 punti]

Solo per questa parte del problema, puoi assumere n (la dimensione di x) uguale a 1, cosicché $\Sigma = [\sigma^2]$ è semplicemente un numero reale, e analogamente il determinante è $|\Sigma| = \sigma^2$. Dato il dataset, affermiamo che le stime di massima verosimiglianza dei parametri sono:

$$\phi = \frac{1}{m} \sum_{i=1}^m \mathbb{1}\{y^{(i)} = 1\} \quad \mu_0 = \frac{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 0\}} \quad \mu_1 = \frac{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^m \mathbb{1}\{y^{(i)} = 1\}}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

La log-verosimiglianza dei dati è

$$\begin{aligned} \ell(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\ &= \log \prod_{i=1}^m p(x^{(i)} \mid y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi). \end{aligned}$$

Massimizzando ℓ rispetto ai quattro parametri, dimostra che le stime di massima verosimiglianza di ϕ , μ_0 , μ_1 e Σ sono effettivamente quelle date dalle formule sopra. (Puoi assumere che ci sia almeno un esempio positivo e uno negativo, cosicché i denominatori nelle definizioni di μ_0 e μ_1 siano diversi da zero.)

(e) [3 punti] Problema di programmazione

In `src/p01e_gda.py`, completa il codice per calcolare ϕ , μ_0 , μ_1 e Σ , usa questi parametri per ricavare θ , e usa il modello GDA risultante per fare predizioni sul validation set.

(f) [5 punti]

Per il Dataset 1, crea un grafico dei dati di training con x_1 sull'asse orizzontale e x_2 sull'asse verticale. Per visualizzare le due classi, usa un simbolo diverso per gli esempi $x^{(i)}$ con $y^{(i)} = 0$ rispetto a quelli con $y^{(i)} = 1$. Sulla stessa figura, traccia il confine di decisione trovato dalla regressione logistica nella parte (b). Crea un grafico identico con il confine di decisione trovato dalla GDA nella parte (e).

(g) [5 punti]

Ripeti i passi della parte (f) per il Dataset 2. Su quale dataset la GDA sembra comportarsi peggio della regressione logistica? Perché potrebbe succedere?

(h) [3 punti extra]

Per il dataset in cui la GDA si è comportata peggio nelle parti (f) e (g), riesci a trovare una trasformazione degli $x^{(i)}$ tale che la GDA funzioni significativamente meglio? Qual è questa trasformazione?

2. [30 punti] Etichette incomplete e solo positive

In questo problema considereremo l'addestramento di classificatori binari in situazioni in cui non abbiamo pieno accesso alle etichette. In particolare, consideriamo uno scenario, non raro nella vita reale, in cui abbiamo etichette solo per un *sottoinsieme* degli esempi positivi. Tutti gli esempi negativi e i restanti esempi positivi sono privi di etichetta.

Assumiamo cioè un dataset $\{(x^{(i)}, t^{(i)}, y^{(i)})\}_{i=1}^m$, dove $t^{(i)} \in \{0, 1\}$ è l'etichetta “vera”, e dove

$$y^{(i)} = \begin{cases} 1 & \text{se } x^{(i)} \text{ è etichettato} \\ 0 & \text{altrimenti.} \end{cases}$$

Tutti gli esempi etichettati sono positivi, ovvero $p(t^{(i)} = 1 \mid y^{(i)} = 1) = 1$, ma gli esempi non etichettati possono essere positivi o negativi. Il nostro obiettivo è costruire un classificatore binario h dell'etichetta vera t , avendo accesso solo alle etichette parziali y . In altre parole, vogliamo costruire h tale che $h(x^{(i)}) \approx p(t^{(i)} = 1 \mid x^{(i)})$ il più fedelmente possibile, usando solo x e y .

Esempio reale: supponi di mantenere un database di proteine coinvolte nella trasmissione di segnali attraverso le membrane. Ogni esempio aggiunto al database è coinvolto in un processo di segnalazione, ma esistono molte proteine coinvolte nella segnalazione trans-membrana che mancano dal database. Sarebbe utile addestrare un classificatore per identificare le proteine da aggiungere. Nella nostra notazione, ogni esempio $x^{(i)}$ corrisponde a una proteina, $y^{(i)} = 1$ se la proteina è nel database e 0 altrimenti, e $t^{(i)} = 1$ se la proteina è coinvolta in un processo di segnalazione trans-membrana (e quindi andrebbe aggiunta al database), 0 altrimenti.

(a) [5 punti]

Supponi che ogni $y^{(i)}$ e $x^{(i)}$ siano condizionatamente indipendenti dato $t^{(i)}$:

$$p(y^{(i)} = 1 \mid t^{(i)} = 1, x^{(i)}) = p(y^{(i)} = 1 \mid t^{(i)} = 1).$$

Nota che questo equivale a dire che gli esempi etichettati sono stati selezionati uniformemente a caso dall'insieme degli esempi positivi. Dimostra che la probabilità che un esempio sia etichettato differisce per un fattore costante dalla probabilità che un esempio sia positivo. Ovvero, mostra che $p(t^{(i)} = 1 \mid x^{(i)}) = p(y^{(i)} = 1 \mid x^{(i)})/\alpha$ per qualche $\alpha \in \mathbb{R}$.

(b) [5 punti]

Supponi di voler stimare α usando un classificatore addestrato h e un validation set V tenuto da parte. Sia V_+ l'insieme degli esempi etichettati (e quindi positivi) in V , dato da $V_+ = \{x^{(i)} \in V \mid y^{(i)} = 1\}$. Assumendo che $h(x^{(i)}) \approx p(y^{(i)} = 1 \mid x^{(i)})$ per tutti gli esempi $x^{(i)}$, mostra che

$$h(x^{(i)}) \approx \alpha \quad \text{per ogni } x^{(i)} \in V_+.$$

Puoi assumere che $p(t^{(i)} = 1 \mid x^{(i)}) \approx 1$ quando $x^{(i)} \in V_+$.

(c) [5 punti] Problema di programmazione

I tre problemi seguenti riguardano un dataset fornito nei file:

`data/ds3_{train,valid,test}.csv`

Ogni file contiene le colonne: x_1 , x_2 , y e t . Come nel Problema 1, c'è un esempio per riga.

Prima consideriamo il caso ideale, in cui abbiamo accesso alle vere etichette t per l'addestramento. In `src/p02cde_posonly`, scrivi un classificatore a regressione logistica che usa x_1 e x_2 come feature di input, e addestralo usando le etichette t (per questa parte puoi ignorare le etichette y). Scrivi le predizioni del modello addestrato sul test set nel file specificato nel codice.

(d) [5 punti] Problema di programmazione

Consideriamo ora il caso in cui le etichette t non sono disponibili, quindi in fase di addestramento hai accesso solo alle etichette y . Aggiungi al tuo codice in `p02cde_posonly.py` il riaddestramento del classificatore (sempre con x_1 e x_2 come feature di input), ma usando solo le etichette y .

(e) [10 punti] Problema di programmazione

Usando il validation set, stima la costante α facendo la media delle predizioni del tuo classificatore su tutti gli esempi etichettati del validation set:

$$\alpha \approx \frac{1}{|V_+|} \sum_{x^{(i)} \in V_+} h(x^{(i)}).$$

Aggiungi codice in `src/p02cde_posonly.py` per riscalarle le predizioni del classificatore della parte (d) usando il valore stimato di α .

Infine, usando una soglia $p(t^{(i)} = 1 | x^{(i)}) = 0,5$, crea tre grafici separati con i confini di decisione delle parti (c)–(e) sovrapposti al test set. Metti x_1 sull'asse orizzontale e x_2 su quello verticale, e usa due simboli diversi per gli esempi positivi ($t^{(i)} = 1$) e negativi ($t^{(i)} = 0$). In ogni grafico, indica l'iperpiano separatore con una linea rossa.

Osservazione: abbiamo visto che la probabilità vera $p(t | x)$ dista solo un fattore costante da $p(y | x)$. Questo significa che, se il compito è solo *ordinare* gli esempi (ad esempio ordinare le proteine dalla più probabile alla meno probabile), allora non serve nemmeno stimare α : l'ordinamento basato su $p(y | x)$ coincide con quello basato su $p(t | x)$.

3. [25 punti] Regressione di Poisson

(a) [5 punti]

Considera la distribuzione di Poisson parametrizzata da λ :

$$p(y; \lambda) = \frac{e^{-\lambda} \lambda^y}{y!}.$$

Mostra che la distribuzione di Poisson appartiene alla famiglia esponenziale, e indica chiaramente i valori di $b(y)$, η , $T(y)$ e $a(\eta)$.

(b) [3 punti]

Considera una regressione con un modello GLM con variabile di risposta di Poisson. Qual è la funzione di risposta canonica della famiglia? (Puoi usare il fatto che una variabile aleatoria di Poisson con parametro λ ha media λ .)

(c) [7 punti]

Per un training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$, sia $\log p(y^{(i)} | x^{(i)}; \theta)$ la log-verosimiglianza di un esempio. Derivando la log-verosimiglianza rispetto a θ_j , ricava la regola di aggiornamento dell'ascesa stocastica del gradiente per un modello GLM con risposte di Poisson y e funzione di risposta canonica.

(d) [7 punti] Problema di programmazione

Considera un sito web che vuole predire il proprio traffico giornaliero. I proprietari del sito hanno raccolto un dataset del traffico passato, con alcune feature che ritengono utili per predire il numero di visitatori al giorno. Il dataset è diviso in train/valid/test e segue lo stesso formato dei Dataset 1–3:

`data/ds4_{train,valid}.csv`

Applicheremo la regressione di Poisson per modellare il numero di visitatori al giorno. Nota che applicare la regressione di Poisson assume che i dati seguano una distribuzione di Poisson il cui parametro naturale è una combinazione lineare delle feature di input (ovvero $\eta = \theta^T x$). In `src/p03d_poisson.py`, implementa la regressione di Poisson per questo dataset e usa l'ascesa del gradiente per massimizzare la log-verosimiglianza di θ .

4. [15 punti] Convessità dei Modelli Lineari Generalizzati

In questa domanda esploreremo e dimostreremo alcune belle proprietà dei Modelli Lineari Generalizzati (GLM), in particolare quelle legate all'uso delle distribuzioni della famiglia esponenziale per modellare l'output.

Più comunemente, i GLM vengono addestrati usando come funzione di perdita la log-verosimiglianza negativa (NLL). Ciò è matematicamente equivalente alla stima di massima verosimiglianza (massimizzare la log-verosimiglianza equivale a minimizzare la log-verosimiglianza negativa). In questo problema, l'obiettivo è mostrare che la perdita NLL di un GLM è una funzione *convessa* rispetto ai parametri del modello. Come promemoria, ciò è conveniente perché in una funzione convessa ogni minimo locale è anche minimo globale.

Ricapitolando, una distribuzione della famiglia esponenziale è una la cui densità di probabilità può essere rappresentata come

$$p(y; \eta) = b(y) \exp(\eta^T T(y) - a(\eta)),$$

dove η è il parametro naturale della distribuzione. Inoltre, in un GLM η è modellato come $\theta^T x$, dove $x \in \mathbb{R}^n$ sono le feature di input dell'esempio e $\theta \in \mathbb{R}^n$ sono i parametri apprendibili. Per mostrare che la perdita NLL è convessa per i GLM, scomponiamo il processo in sottoparti e le affrontiamo una alla volta. L'approccio è mostrare che la derivata seconda (cioè l'Hessiana) della perdita rispetto ai parametri è semidefinita positiva (PSD) per ogni valore dei parametri. Lungo la strada dimostreremo anche alcune belle proprietà delle distribuzioni della famiglia esponenziale.

Per comodità ci restringiamo al caso in cui η è uno scalare. Assumi $p(Y | X; \theta) \sim \text{FamigliaEsponenziale}(\eta)$, dove $\eta \in \mathbb{R}$ è uno scalare e $T(y) = y$. La rappresentazione diventa quindi

$$p(y; \eta) = b(y) \exp(\eta y - a(\eta)).$$

(a) [5 punti]

Ricava un'espressione per la media della distribuzione. Mostra che $\mathbb{E}[Y | X; \theta]$ può essere rappresentata come il gradiente della funzione di log-partizione a rispetto al parametro naturale η .

Suggerimento: parti dall'osservazione che $\frac{\partial}{\partial \eta} \int p(y; \eta) dy = \int \frac{\partial}{\partial \eta} p(y; \eta) dy$.

(b) [5 punti]

Poi, ricava un'espressione per la varianza della distribuzione. In particolare, mostra che $\text{Var}(Y | X; \theta)$ può essere espressa come la derivata della media rispetto a η (cioè la derivata seconda della funzione di log-partizione $a(\eta)$ rispetto al parametro naturale η).

(c) [5 punti]

Infine, scrivi la funzione di perdita $\ell(\theta)$, la NLL della distribuzione, come funzione di θ . Poi calcola l'Hessiana della perdita rispetto a θ e mostra che è sempre PSD. Questo conclude la dimostrazione che la perdita NLL dei GLM è convessa.

Suggerimento: usa la regola della catena insieme ai risultati delle parti precedenti per semplificare le derivazioni.

Osservazione: i punti chiave di questo problema sono:

- Ogni modello GLM è convesso nei suoi parametri.

- La famiglia esponenziale di distribuzioni è matematicamente “comoda”: mentre calcolare media e varianza di una distribuzione in generale richiede integrali (difficile), sorprendentemente per la famiglia esponenziale possiamo calcolarle con delle derivate (facile).

5. [25 punti] Regressione lineare pesata localmente

(a) [10 punti]

Considera un problema di regressione lineare in cui vogliamo “pesare” diversamente i vari esempi di training. In particolare, vogliamo minimizzare

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m w^{(i)} (\theta^T x^{(i)} - y^{(i)})^2.$$

A lezione abbiamo visto cosa succede nel caso in cui tutti i pesi ($w^{(i)}$) sono uguali. In questo problema generalizziamo alcune di quelle idee al caso pesato.

- i. [2 punti] Mostra che $J(\theta)$ può anche essere scritta come

$$J(\theta) = (X\theta - y)^T W (X\theta - y)$$

per un’opportuna matrice W , dove X e y sono definiti come a lezione. Specifica chiaramente il valore di ogni elemento della matrice W .

- ii. [4 punti] Se tutti i $w^{(i)}$ valgono 1, abbiamo visto a lezione che l’equazione normale è

$$X^T X \theta = X^T y,$$

e che il valore di θ che minimizza $J(\theta)$ è $(X^T X)^{-1} X^T y$. Calcolando la derivata $\nabla_{\theta} J(\theta)$ e ponendola uguale a zero, generalizza l’equazione normale al caso pesato, e fornisci il nuovo valore di θ che minimizza $J(\theta)$ in forma chiusa, come funzione di X , W e y .

- iii. [4 punti] Supponi di avere un dataset $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$ di m esempi indipendenti, ma di modellare gli $y^{(i)}$ come estratti da distribuzioni condizionali con livelli di varianza diversi $(\sigma^{(i)})^2$. In particolare, assumi il modello

$$p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi} \sigma^{(i)}} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2(\sigma^{(i)})^2}\right).$$

Ovvero, ogni $y^{(i)}$ è estratto da una gaussiana con media $\theta^T x^{(i)}$ e varianza $(\sigma^{(i)})^2$ (dove i $\sigma^{(i)}$ sono costanti fisse e note). Mostra che trovare la stima di massima verosimiglianza di θ si riduce a risolvere un problema di regressione lineare *pesata*. Indica chiaramente quali sono i $w^{(i)}$ in funzione dei $\sigma^{(i)}$.

(b) [10 punti] Problema di programmazione

Consideriamo ora il seguente dataset (il formato coincide con quello dei Dataset 1–4, tranne che $x^{(i)}$ è unidimensionale):

`data/ds5_{train,valid,test}.csv`

In `src/p05b_lwr.py`, implementa la regressione lineare pesata localmente usando le equazioni normali ricavate nella Parte (a) e usando

$$w^{(i)} = \exp\left(-\frac{\|x^{(i)} - x\|_2^2}{2\tau^2}\right).$$

Addestra il modello sul train split usando $\tau = 0,5$, poi esegui il modello sul valid split e riporta l’errore quadratico medio (MSE). Infine traccia le predizioni del modello sul validation set (traccia il training set con marker ‘x’ blu e il validation set con marker ‘o’ rossi). Il modello sembra essere in underfitting o in overfitting?

(c) [5 punti] Problema di programmazione

Ora ottimizziamo l'iperparametro τ . In `src/p05c_tau.py`, trova l'MSE del modello sul validation set per ciascuno dei valori di τ specificati nel codice. Per ogni τ , traccia le predizioni del modello sul validation set nel formato descritto nella parte (b). Riporta il valore di τ che ottiene l'MSE più basso sul valid split, e infine riporta l'MSE sul test split usando questo valore di τ .